Python

Irene Siragusa, PhD

Tools

- VSCode
  - https://code.visualstudio.com/download
- Python ≥ 3.7
  - [W10-W11] Microsoft store
  - [Linux] packet manager (apt, snap, …)
  - [MacOS] brew
  - [all] https://www.python.org/downloads/
- Google Colab
  - https://colab.google/

Python

- Python is an interpreted, general-purpose, object-oriented language that is widely used nowadays for data analysis and machine learning applications.

- Syntax and indentation (tabs) are important to define your code

```
# comment
""" multiline
comments """
```

Python – locally

- Create folder big data
- Test via termial per python
  - py windows installed via url
  - python windows from store
  - python/python3 mac
- Create venv

```
python3 -m venv <path/to/env>
```

- Activate venv

[linux/mac] `source <path/to/env>/bin/activate`
[windows] `<path\to\env>\Scripts\activate.bat`

- You can also create in a folder via VSCode

Python

How execute the code?

- Terminal
- digit python (or python3)
- Read → Eval → Print → Loop
- Each expression is read, evaluated and the output is directly printed

```
$ python
Python 3.12.0 (main, ...)
>>> 2 + 3
5
>>> name = 'Alice'
>>> print(f'Hello, {name}!')
Hello, Alice!
>>> exit() # or Ctrl+D / Ctrl+Z
```

Python

How execute the code?

- Script from terminal

python [file_name.py]

hello.py

```
import sys

if __name__ == '__main__':
    if len(sys.argv) > 1:
        name = sys.argv[1]
        print(f'Hello {name}!')
    else:
        print(f'Hello world!')
```

```
$ python hello.py
Hello world!
$ python hello.py Alice
Hello Alice!
```

Python

How execute the code?

- Notebooks
- File divided into blocks executed separately

hello.ipynb

```
print(f'Hello world!')

>>> Hello world!


name = 'Alice'
print(f'Hello, {name}!')
>>> Hello, Alice!
```

Data types

- Integers
  - Represented with 32-bits, can be converted from others formats with `int()`
- Reals
  - Can be obtained via built-in function `float()` with range [-10^{308} , 10^{308}] a number can be printed as -3.456 or in scientific notation -3.456e-7
- Booleans
  - Related to logical operations and to relational operator and can have only True or False values

Data types

Type	Examples	Note
`int`	`42, -7, 0xFF, 0b1010`	arbitrary precision
`float`	`3.14, 1e-3, float('inf')`	IEEE 754 double (64 bit)
`complex`	`2+3j, complex(1,-1)`	real part + imaginary part
`bool`	`True, False`	subclass of int (1/0)
`bytes`	`b'abc', bytes(4)`	immutable byte sequence
`NoneType`	`None`	No value

Data types – examples

```
n = 42

pi = 3.14159

ok = True

type(n) # <class 'int'>

isinstance(n, int) # True
```

Data types – examples

- To output/display a python object use `print()` function
- To print well-formatted strings, use `print(f'')`
  - `pi = 3.141592653589793`
  - `print(f'Pi is {pi:1.5f}') -> Pi is 3.14159`

Data types – strings

- Strings can be obtained via `str()` or within single or double apex notation
- Escape characters are allowed

  `\n, \t, \b, \\, \', \”`
- Can be concatenated via + operator
- Cannot be modified, if manipulation is needed a new object is built.
- A sequential access can be performed with a direct positional indexing from zero.

Data types – strings

- `s.lower()` and `s.upper()` return string s in lower/upped case
- `s.strip()` removes any white spaces at the beginning and at the end of the string
- `s.isalpha()`, `s.isdigit()`, `s.isspace()` returns `True` if ALL chatacters in a string are alphanumeric, numeric, spaces
- `s.startswith('other')` and `s.endswith('other')` check if string `s` starts/ends with string `other`

Data types – strings

- `s.find('other')` search other in s and returns the starting index of the first occurrence, otherwise -1
- `s.replace('old', 'new')` returns a string where all the occurrences of `old` are substituted with `new`
- `s.split(delim)` return a list of sub-strings divided by the given delimitator. Default delimitator is a single space.
- `s.join(list)` concats element of a given `list` in a string with `s` as delimitator.

Data types – lists

- List can be defined as `a = [element, element, …]`
- Can contain different data types of elements simultaneously
- Can be accessed via array notation and slicing
- Can be modified with in-place operations

Data types – lists

- `len(a)` return length of a given list
- `a.pop(0)` removes first element
- `a.append('Monica')` appends given element at the end of a list
- `a.insert(1, 'Joey')` insert given element at given position
- `list.extend(['Rachel', 'Phoebe'])` extends the given list with another
- `print(list.index('Joey'))` returns index of a given element
- `list.remove('Joey')` if the element is found, is removed from a list
- `print(list[1:-1])` prints list from index 1 to second-to-last element

Data types – tuple

- Tuple can be defined as `a = (element, element, …)`
- Can contain different data types of elements simultaneously
- Are ordered and immutable

Data types – sets

- Sets can be defined as `a = {element1, element2, …}`
- Can contain different data types of elements simultaneously
- Automatic deduplication
- Support native algebraic operations as union, intersection, difference

Data types – dictionaries

- Dictionaries can be defined as `d = {k1: v1, k2: v2}`
- Are a key-value hash tables
- Keys must be immutable types, no restrictions for values types

Data types

Type	Examples	Note
`str`	`'bye', "hello", r'\n'`	immutable, ordered
`list`	`[1, 'a', 3.0]`	ordered, allows duplicates
`tuple`	`(1, 'a', 3.0) or 1`	immutabile, ordered
`set`	`{1, 2, 3} or set()`	unordered, no duplicates
`dict`	`{'k': v, 1: [2,3]}`	key-value pairs, ordered

Data types – examples

```
msg = 'Python 3' # string

lst = [10, 20, 30, 10] # list

tpl = (10, 20, 30) # tuple

st = {10, 20, 30} # set

d = {'name': 'Anna', 'age': 25}
d['name']  # value access
```

Data types – conversions

```
int('42')           # 42
float('3.14')       # 3.14
str(100)            # '100'
bool(0)             # False (0, None, [] → False)
list('abc')         # ['a', 'b', 'c']
tuple([1,2,3])      # (1, 2, 3)
set([1,1,2,3])      # {1, 2, 3}
dict(a=1, b=2)      # {'a': 1, 'b': 2}
```

Data types – slicing and indexing

- Strings, list and tuples can be accessed with the following notation `[start:stop:step]`
  - start -> inclusive first index
  - stop -> exclusive upper limit
  - step -> stride increment size
- Their length can be obtained via `len()` function
- Note that stop position is excluded (you arrive at len-1)
- You can use a variant of the standard indexing

Data types – slicing and indexing

```
a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

a[1:4]
 >>> [1, 2, 3]

a[1:6:2]
 >>> [1, 3, 5]

a[::-1]
 >>> [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Operators

Sorting

- Sorting of an iterable object can be obtained using the following function

```
sorted(iterable, key=None, reverse=False)
```

- It return a list of ordered element in ascending order.
  - `key` refers to a function that works as ordering criteria
  - `reverse=True` enables descending ordering.
- Lists have the built-in method `sort()` for in place ascending sorting and returns `None`

E1. String manipulation

Given s = " Big Data Tools, Python and Big Data Analytics ":

- Remove the surrounding whitespace and convert to lower and upper case.
- Check whether it starts with "big" and ends with "analytics".
- Find the index of the first and the second "Big»
  - *hint: find accepts a start position.*
- Replace "Big Data" with "BD".
- Split the string by "," and rejoin the pieces using " | ".

E2. List manipulation

Start from a = [‘Raj', ‘Penny', 'Sheldon’]

- Obtain [‘Bernardette', ‘Penny', 'Howard', 'Amy', 'Sheldon’], using pop, append, insert, extend, remove, and index at least once each.
- Print the list reversed using slicing, and every element with an even index.

Control functions – `if elif else`

```
if condition:
      istruction
elif condition:
      istruction
    ...
else:
    istruction
```

```
x = 7
if x < 0:
      print('negative')
elif x == 0:
      print('zero')
else:
      print('positive')


# ternary condition
sign = 'pos' if x > 0 else 'non-pos'
```

Control functions – `for`

```
for iter in iterable:
    istruction
```

```
for i in range(5): # 0,1,2,3,4
    print(i)

# iterates over iterable objects
# string, tuple, sets
for fruit in ['apple','pear','kiwi']:
    print(fruit)
```

Control functions – `for`

```
for iter in iterable:
    istruction
```

```
# enumerate: index + value
for i, v in enumerate(['a','b','c']):
    print(i, v) # 0 a, 1 b, 2 c

# items: for dictionary
dd = dict(z=-1, y=-2)
for k, v in dd.items():
    print(k, v) # z -1, y -2
```

Control functions – `for`

```
for iter in iterable:
    istruction
```

```
# break / continue / else
for n in range(10):
    if n == 5:
        break
    if n % 2:
        continue
    else:
        print('completed')
```

Control functions – List Comprehension

The python-way to create dynamically a list

```
newlist = [expression for item in iterable]
```

```
# classic way
fruits = ["banana", "cherry", "kiwi", "lemom", "mango"]
newlist = []

for x in fruits:
        newlist.append('* '+x)
print(newlist)

# list comprehension
newlist_2 = ['* '+x for x in fruits]
print(newlist_2)
```

Control functions – List Comprehension

```
newlist = [expression for item in iterable if
 condition == True]
```

```
# classic way
newlist = []
#enumerate outputs the index and the value
for idx, x in enumerate(fruits):
        if idx%2==0:
                newlist.append(x)
print(newlist)
# list comprehension
newlist_2 = [x for idx, x in enumerate(fruits) if idx%2==0]
print(newlist_2)
```

Control functions – `Dictionary Comprehension`

The python-way to create dynamically a list

```
newlist = [expression for item in iterable if
  condition == True]
```

```
dict1 = {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}

dict2 = {}
for k, v in dict1.items():
        dict2[k] = v**2
print(dict2)

# dict comprehension
dict3 = {k:v**2 for k,v in dict1.items()}
print(dict3)
```

Control functions – `while`

```
while condition:
    istruction
```

```
n = 1
while n < 100:
    n *= 2
    print(n) # 128
```

Control functions – `match/case`

```
match variable:
        case value_1:
            instruction
        case value_2:
            instruction
        case _:
            instruction
```

```
command = 'quit'
match command:
        case 'quit' | 'exit':
                print('exit program…')
        case 'help':
                print('Help mode…')
        case _:
                print('Unknown command')
```

Python 3.10+

Control functions – `try/except/finally`

```
try:
    except erroType:
        instruction
    else:
        instruction
    finally:
        instruction
```

```
try:
    x = int(input('Number: '))
    print(10 / x)
    except ValueError:
       print('Invalid input')
    except ZeroDivisionError:
       print('Cannot divde per zero')
    except Exception as e:
      print(f'Error: {e}')
    else:
      print('No errors')
    finally:
      print('Always print')
```

Functions

```
def say_hello(name, times):
    """say hello multiple times."""
    for _ in range(times):
        print(f'Hello, {name}!')


say_hello('Anna', 2) # order counts
say_hello(times=3, name='Bob') # keyword-based
```

Functions – default and optional values

```
def power(base, esp=2, *, verbose=False):
    # parameters after * are only keyword
    result = base ** esp
    if verbose:
        print(f'{base}^{esp} = {result}')
    return result
power(3) # 9
power(2, 10) # 1024
power(2, verbose=True) # 2^2 = 4
```

Functions – lambda functions

```
square = lambda x: x ** 2
square(5) # 25


# useful with sorted, map, filter
data = [('Bob',30),('Anna',25),('Luca',35)]
sorted(data, key=lambda t: t[1]) # age sorting
```

Functions – *args and **kwargs

```
def num_sum(*numbers): # args: tuple
    return sum(numbers)
num_sum(1, 2, 3) # 6
num_sum(1, 2, 3, 4) # 10

def show_f(**info): # kwargs: dictionary
    for k, v in info.items():
    print(f'{k}: {v}')
show_f(name='Anna', eta=25, citta='PA')
```

Functions – unpacking

```
def vectors(x, y, z):
    return x + y + z


coords = [1, 2, 3]
vectors(*coords) # unpacking list
params = {'x':1,'y':2,'z':3}
vectors(**params) # unpacking dictionary
```

Functions – parameter order

Parameter order in Python Function Signature

```
def f(pos1, pos2, *args, kw1='x', **kwargs):
    pass
```

*args -> positional values, as tuple

*kwargs -> key-value pairs, as dictionary

positionals values / default and/or optional arguments / * args / keyword-only / **kwargs

Class

- Each item can be modelled as an object with attributes and methods (object's functions)

```
class Person:
    def __init__(self, name, age):
        self.name = name # attribute
        self.age = age

    def greet(self):
        return f"Hi! I am {self.name}."

    def __str__(self):
        return f"Person({self.name}, {self.age})"
```

```
user = Person("Alice", 30)
print(user.greet()) # Hi! I am Alice.
print(user) # Person(Alice, 30)
```

Import

- Import of the entire module or library

```
import math
math.sqrt(16) # namespace access
```

- Import w/ alias

```
import numpy as np
np.array([1,2,3])
```

Import

- Import of specific names

```
from os.path import join, exists
join('/home', 'user') # w/o prefix
```

- Import all (bad, may overrided names)

```
from math import *
sqrt(4)
```

Input/Output operations

- Keyword *with* delimits a code block inside which the file is opened and can be both written or read, depending the opening mode

```
with open("data.txt", mode="r", encoding="utf-8") as f:
    content = f.read()
# "r" = read | "w" = overwrite | "a" = append | "r+" =
 read/write
```

Input/Output operations

- Writing

```
with open("log.txt", "w", encoding="utf-8") as f:
    f.write("Line 1: Server OK\n")
```

- Reading

```
with open("log.txt", "r", encoding="utf-8") as f:
    content = f.readlines() # reading line by line
for line in content:
    print(line)
```

Input/Output operations – JSON

JSON (JavaScript Object Notation) is a dictionary-like data format

```
import json
data = {"user": "Alice", "role": "Admin"}

# Save dict as JSON file
with open("user.json", "w") as f:
    json.dump(data, f, indent=2)

# Read JSON file into dict
with open("user.json", "r") as f:
    loaded = json.load(f)
```

Input/Output operations – CSV

CSV (Comma Separated Values) is a tab-like data format

```
import csv

# Writing dictionary rows to CSV
with open("items.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["id", "name", "price"])
    writer.writerow([101, "Book", 19.99])

# Reading rows back
with open("items.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"])
```

Python

- Python 3 docs: https://docs.python.org/3/
- Tutorial: https://docs.python.org/3/tutorial/
- Built-in Functions: https://docs.python.org/3/library/functions.html
- The Python Standard Library: https://docs.python.org/3/library/index.html

Python – Useful resources

- Google's Python Class: https://developers.google.com/edu/python
- Google's Python Class exercises: https://developers.google.com/edu/python/google-python-exercises.zip
- W3Schools Python: https://www.w3schools.com/python/

E3. Statistical measures

- Calculate mean, variance and standard deviations of a given list of numbers

- Double check the obtained results with native functions in statistics module

E4. Degrees to Radians

- Implement a radians function that returns the radian equivalent of a degree. Use the following formula:

```
rad = degrees * pi/180
```

- Use this function to print the radian equivalent of all degrees ranging from 1° to 180°. Use two digits of precision for the results.

E5. Duplicate Elimination

- In organizations, a list of email addresses is often compiled for marketing purposes. However, duplicate email addresses need to be removed from this list. Write a function that receives a list and returns a list containing only unique values. Test your function with a list of email addresses.

E6. Counting Votes

- Given the votes, arranged as a string where each vote is separated from the next by a comma, write a script that uses a dictionary to determine the number of votes received by candidates in an election.

E7. Class Average – TXT

- Write code that enables you to store any number of grades into a grades.txt plain text file.
- Write code that reads the grades from the grades.txt file you created in the previous exercise.
- Display the individual grades and their total, count and average.

E8. Class Average – CSV

- An instructor teaches a class in which each student takes three exams. The instructor would like to store this information in a file named grades.csv. Write code that enables an instructor to enter each student's first name and last name as strings and the student's three exam grades as integers.
- Use the csv module to write each record into the grades.csv file. Each record should be a single line of text in the following CSV format:

```
firstname,lastname,exam1grade,exam2grade,exam3grade
```

- Read the grades.csv and display the data in tabular format.

E9. Class Average – JSON

- Use the json module to write the student information to the file in JSON format, create a dictionary of student data in the following format:

```
gradebook_dict = {'students': [student1dictionary,
                  student2dictionary, ...]}
```

- Each dictionary in the list represents one student and contains the keys 'first_name', 'last_name', 'exam1', 'exam2' and 'exam3', which map to the values representing each student's first name (string), last name (string) and three exam scores (integers).
- Output the gradebook_dict in JSON format to the file grades.json.
- Read the grades.json and display the data in tabular format.